Methodology and Findings

Chapters 3 and 4 of the bachelor thesis "Latency and Accuracy Optimization for OCR to LLM Document Processing Pipelines"

Fleming Fredy Silva · [University name] · 2026

Contents

3 Methodology 2

3.1 Research approach 2

3.2 Research design and variables 3

3.3 Systems under test 4

3.4 Dataset and ground truth 5

3.5 Measurement instruments and metrics 6

3.6 Experimental procedure 7

3.7 Additional experiments 8

3.8 Data analysis 8

3.9 Validity, reliability and ethics 9

4 Findings 10

4.1 Overview 10

4.2 Performance of Architecture A and Architecture B (RQ1) 10

4.3 Accuracy of Architecture A and Architecture B (RQ1) 12

4.4 Sources of the speed-up (RQ2) 14

4.5 Model precision and quantisation (RQ2) 15

4.6 Routing and cache (RQ2) 16

4.7 OCR engine (RQ3) 17

4.8 Vision-language model on the same GPU (RQ3) 17

4.9 Further findings on data quality and measurement 18

4.10 Summary of findings by research question 19

References 19

3 Methodology

This chapter explains how the research was carried out: the research approach and design (Sections 3.1 and 3.2), the two systems under test (Section 3.3), the dataset and its ground truth (Section 3.4), the measurement instruments and metrics (Section 3.5), the experimental procedure (Section 3.6), the additional experiments (Section 3.7), the data analysis (Section 3.8), and the measures taken for validity, reliability and ethics (Section 3.9). The aim is that another researcher with the same hardware could repeat every measurement reported in Chapter 4.

3.1 Research approach

The thesis follows a **design science** approach (Hevner et al., 2004): it builds artefacts that address a practical problem, two OCR → LLM pipeline architectures, and evaluates them rigorously against defined criteria. This matches the thesis type, a solution to a current problem. The evaluation is **quantitative and experimental**. Both artefacts process the same documents under controlled conditions, every configuration is repeated, and the outcomes are measured with instruments that are part of the implementation.

Three research questions guide the work. Table 3.1 shows how each is answered.

- **RQ1.** By how much can an asynchronous architecture with parallel OCR, batching, caching and a quantised model improve throughput, latency, memory and cost compared with a sequential baseline using the same models and hardware, and does accuracy stay the same?
- **RQ2.** Which of these optimisations account for the improvement, and which do not pay off at this scale?
- **RQ3.** On the same consumer GPU, how does the OCR → LLM approach compare with a vision-language model (VLM) that reads the invoice image directly?

Table 3.1. Research questions, the experiments that answer them, and the measures used.

RQ	Experiment	Measures
RQ1	Main benchmark: A vs B on 600 invoices, 3 runs; latency experiment with random arrivals	throughput, wall time, latency (median, p95), memory, cost, F1
RQ2	Ablation: nine variants of B, each changing one factor; routing-threshold analysis; warm/cold cache runs	wall time, stage times, GPU memory, F1, routing shares
RQ3	OCR-engine comparison (Tesseract vs EasyOCR); VLM baseline (Qwen2-VL-2B)	F1, fully correct invoices, time per invoice, GPU memory

The research proceeded in six steps (Figure 3.1). The analysis of the literature and of the first prototype (step 1) produced the requirements. The two architectures were then designed and built with a shared extraction stage (step 2), and the dataset was prepared (step 3). The benchmark (step 4) and the additional experiments (step 5) produced the measurements that were analysed in step 6. The process included one deliberate iteration: the error analysis of the first complete benchmark showed that most wrong totals were caused by the extraction stage that both architectures share. That stage was improved (row-ordered OCR text and an arithmetic total check), and the whole benchmark was run again. Only the results of the final run are used to compare the architectures; the first run is kept as a record of the improvement.

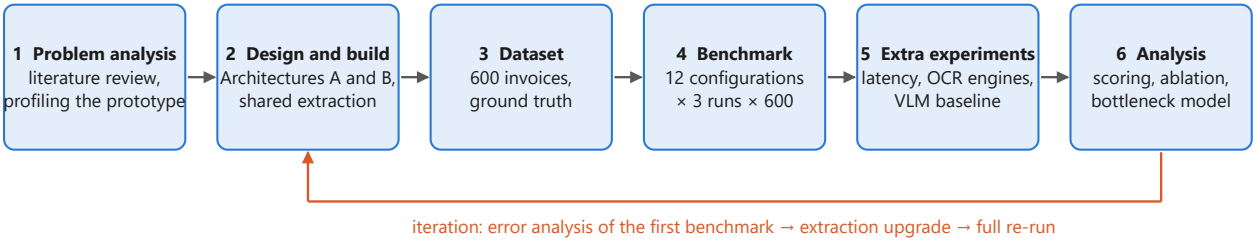


Figure 3.1. Research process. The orange loop is the single iteration: an error analysis of the first benchmark led to an improvement of the shared extraction stage and a complete re-run.

3.2 Research design and variables

The core of the design is a **controlled comparison**. Architectures A and B use exactly the same OCR engine and settings, line filter, language model, prompts, post-processing, validation and scorer, and both run their models on the same GPU. Any difference in time, memory or cost can therefore be attributed to the execution architecture, and any difference in accuracy can come only from the numeric precision of the model on B's fast route. The comparison is extended by a **one-factor-at-a-time ablation**: each variant of B changes exactly one factor, so comparing it with B isolates the contribution of that factor. Table 3.2 lists the variables.

Table 3.2. Variables of the experiments.

Type	Variables
Independent	architecture (A, B); in B: number of OCR workers (1, 3, 6), batch size (1, 8), cache (off, cold, warm), model precision (fp32, fp16, 8-bit, 4-bit), routing policy (initial thresholds, derived thresholds, all-FAST, all-FULL); in the additional experiments: arrival rate (30–240 invoices/min), batching policy (fixed, dynamic), OCR engine (Tesseract, EasyOCR), reader (OCR pipeline, VLM)
Dependent	F1 overall, per field and per subset; share of fully correct invoices; wall time; throughput; service time; latency (median, p95); OCR, line-filter and LLM time per invoice; peak RAM; GPU memory; cost per 100 invoices; straight-through rate; accuracy of auto-accepted invoices
Controlled	models and weights, prompts, post-processing, scorer and thresholds, dataset and its order, hardware, operating system and library versions, warm-up, offline model loading, power settings, absence of other workloads

Table 3.3. Benchmark configurations. Each variant of B changes one factor.

Configuration	What changes	What it isolates
A	sequential baseline, fp32	reference
B	all optimisations; cache cleared before each run	the optimised architecture on a first pass
B-warm	B repeated immediately with a full cache	steady-state benefit of the cache
B-nocache	no cache	cost of the cache lookup
B-allfull	every invoice on the fp32 model	B without the quantised model
B-allfast	every invoice on the 4-bit model	B with only the quantised model
B-fp16	every invoice on an fp16 model	half precision instead of quantisation
B-8bit	every invoice on an LLM.int8 model (Dettmers et al., 2022)	8-bit instead of 4-bit quantisation
B-1worker	1 OCR worker instead of 3	value of OCR parallelism
B-6workers	6 OCR workers	how far OCR parallelism scales
B-nobatch	batch size 1 instead of 8	value of GPU batching
B-tuned	routing thresholds derived from B-allfast and B-allfull	data-derived routing

3.3 Systems under test

The two systems are described in detail in Chapter 2; this section summarises what is needed to interpret the measurements.

Architecture A (Figure 3.2) processes one invoice at a time in one process. Tesseract 5 (Smith, 2007) reads the image on the CPU, and the OCR text is rebuilt row by row from the word positions. A MiniLM sentence-embedding filter (Reimers and Gurevych, 2019; Wang et al., 2020) then keeps the lines most similar to "invoice number", "invoice date" and "total amount due", plus the line after each. Flan-T5-base (Chung et al., 2022) in fp32 answers one question per field in three separate calls. Finally, the answers are normalised, the total is reconciled arithmetically (subtotal + tax = total), and a review flag is set if a field is missing or unparsable, the total is not on a total line, or OCR confidence is below 80.

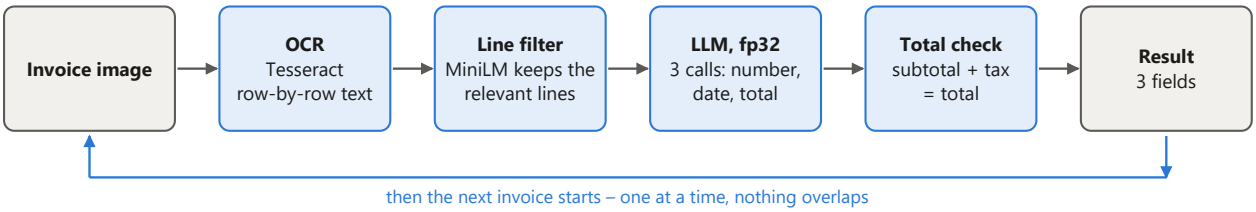


Figure 3.2. System under test 1: Architecture A, sequential.

Architecture B (Figure 3.3) looks up each invoice in a SQLite cache keyed by the SHA-256 digest of the configuration and the image bytes. For the remaining invoices, an asyncio event loop sends OCR jobs to three worker processes and hands every 8 finished results as a batch to a single GPU thread. The GPU thread filters the lines of the whole batch in one call and routes each invoice: to the FAST route, a 4-bit NF4 copy of Flan-T5 (Dettmers et al., 2023) with at most 12 answer tokens, if its OCR text has at most T tokens and its OCR confidence is at least C; otherwise to the FULL route, the fp32 copy with at most 24 tokens. It then answers all questions of a route in one call with up to 24 prompts, and applies the same reconciliation and review flag as A.

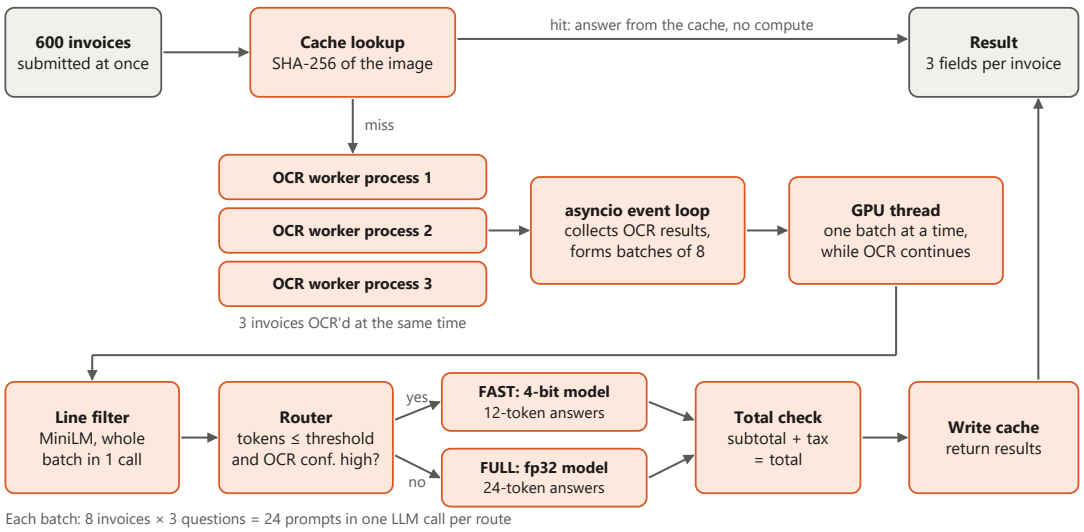


Figure 3.3. System under test 2: Architecture B, asynchronous with cache, parallel OCR, batching and routing.

Table 3.4. Fixed parameters of the systems under test.

Parameter	Value
OCR engine and layout	Tesseract 5.4, `image_to_data`, greyscale, text rebuilt row by row (tolerance 0.6 × median line height)
Line filter	all-MiniLM-L6-v2; 6 lines per query plus the following line; texts of ≤ 18 lines unchanged
Language model	google/flan-t5-base (248 M parameters), greedy decoding, input ≤ 512 tokens
4-bit configuration	bitsandbytes NF4, double quantisation, fp16 compute
B: OCR workers, batch size	3 processes, 8 invoices (24 prompts per call)
B: routing thresholds	initial T = 300 tokens, C = 85; derived T = 525, C = 70 (B-tuned)
B: answer length	FAST ≤ 12 new tokens, FULL ≤ 24 new tokens
Review flag	missing or unparsable field, total not on a total line, OCR confidence < 80

3.4 Dataset and ground truth

The dataset consists of 600 single-page invoice images, each paired with a JSON ground-truth file holding the invoice number, the invoice date and the total amount due (Table 3.5, Figure 3.4).

Table 3.5. Composition of the dataset.

Subset	Invoices	Origin	Image formats
Real	1–15 and 316–600 (300)	publicly available invoice templates and scanned invoices, labelled by hand	WebP, PNG, JPG; 1131 × 1600 to 2481 × 3509 px
Synthetic	16–315 (300)	rendered by a generator: company name, invoice number, date, optional line items and distracting lines, total	PNG, 800 × 600 or 800 × 1000 px

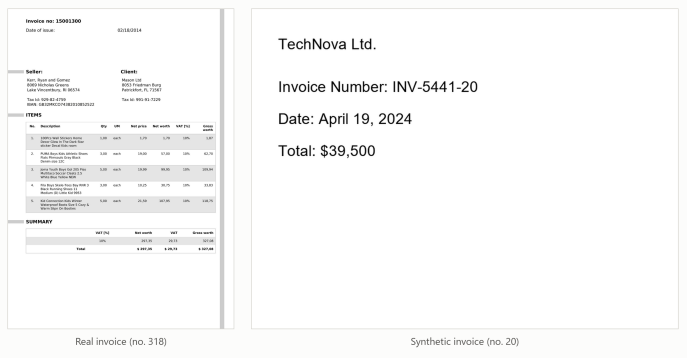


Figure 3.4. A real invoice (left), whose summary row holds net, tax and gross side by side, and a synthetic invoice (right).

The real invoices vary in layout, fonts, label wording, currency notation and scan quality; many contain line-item tables and a summary row with the net amount, the tax and the gross amount next to each other. The synthetic invoices are clean, in one font, with labelled fields. They serve as a sanity check, since a working pipeline should reach almost 100 % on them, while the real invoices are the actual test. Every accuracy figure is therefore reported for the two subsets separately.

Ground-truth labelling and correction. Labels were written by hand in the format printed on the invoice. The total is defined as the gross amount due. During the error analysis it turned out that 26 labels of real invoices held the net amount instead. A script located the net, tax and gross amounts in each invoice's OCR text and changed a label from net to gross only when net + tax = gross held to the cent; labels without such a consistent summary were left unchanged. Every decision was logged and the original files were kept. To prevent the corrections from inflating the results, the accuracy improvement was also analysed on the 574 labels that were never changed (Section 4.3).

Documents are processed in their numeric order (1, 2, …, 600), and a run refuses to start if any ground-truth file is missing. Both rules were introduced after the first prototype had sorted files as strings and silently skipped one invoice.

3.5 Measurement instruments and metrics

3.5.1 Accuracy

Each predicted field is normalised and compared with its ground truth: invoice numbers to upper-case alphanumerics, dates to calendar dates, amounts to numbers. A correct value is a true positive (TP), and a missing value a false negative (FN). A wrong value counts as both a false positive (FP) and a false negative, because the system asserted something false and missed the true value. A value predicted where the ground truth is empty is an FP. The matching rules are:

- **Invoice number:** equal canonical strings, equal digit sequences (at least three digits), or a RapidFuzz similarity ratio of at least 90, which tolerates one OCR character error in a ten-character number and rejects two.
- **Date:** the same calendar day, with day and month allowed to swap, because numeric dates such as 07/06/2025 are ambiguous in the ground truth itself.
- **Total:** a numeric difference of less than 0.005. Amounts are not matched fuzzily, because `6,270` and `6,999` look similar as strings but are different amounts.

Precision, recall and F1 are micro-averaged over all fields (Manning et al., 2008):

$$P = TP / (TP + FP),\ R = TP / (TP + FN),\ F1 = 2 \cdot P \cdot R / (P + R) \tag{3.1}$$

The share of invoices with all three fields correct (*fully correct*) is reported as a user-oriented measure. For the review flag three measures are reported: the *straight-through rate* (share of invoices accepted without

review), the share of accepted invoices that are fully correct, and the share of wrong invoices that the flag sends to a person.

3.5.2 Performance and cost

Table 3.6. Performance metrics and how they are measured.

Metric	Definition and instrument
Wall time	time from submitting the first invoice to receiving the last result (`time.perf_counter`), models loaded and warmed up
Throughput	invoices ÷ wall time × 60
Service time	compute spent on one invoice: OCR time + its share of the batch's line-filter and model time; equal to latency in A
Latency	time from submission (or arrival) to result, including waiting; reported as median (p50) and 95th percentile (p95)
Stage times	mean OCR, line-filter and LLM time per uncached invoice
Peak RAM	maximum resident memory of the whole process tree (main process and OCR workers), sampled 4 times per second with `psutil`
Pipeline GPU memory	PyTorch's peak allocated GPU memory during the run
Whole-GPU memory	maximum memory in use on the card (NVML), including other programs, as a check that the 4 GB card is not exceeded
Cost	modelled from wall time (Equation 3.2)

$$\textit{cost per 100 invoices} = 100 \cdot (\textit{wall time} / N) \cdot €0.35 \textit{ per GPU-hour} / 3600\ s \tag{3.2}$$

The price of €0.35 per GPU-hour is a typical on-demand price for an entry-level cloud GPU. Because cost is proportional to wall time, the ratio between the architectures does not depend on the chosen price. Model loading and warm-up are timed separately and excluded, since they are paid once per process and not per invoice.

3.6 Experimental procedure

All measurements were taken on one laptop: an NVIDIA GeForce RTX 3050 Laptop GPU with 4 GB, an Intel Core 12th-generation CPU with 12 logical cores and 15.7 GB of RAM, running Windows 11, Python 3.12.2, PyTorch 2.5.1 with CUDA 12.1, transformers 5.7.0, bitsandbytes 0.50.2, sentence-transformers 6.1.0, RapidFuzz 3.14.6 and Tesseract 5.4.0. The benchmark followed a fixed protocol:

1. **Preparation.** The laptop was plugged in, sleep was disabled by the benchmark runner, other programs were closed, and models were loaded from the local cache in offline mode.
2. **Warm-up.** Each architecture processed one dummy invoice before timing started, so that the first real invoice did not pay GPU initialisation.
3. **Runs.** Each configuration processed all 600 invoices three times. Before every cold run the result cache was cleared; B-warm ran immediately after B on the full cache. Each run wrote one CSV row per invoice (timings, route, OCR confidence, token count, prediction, ground truth, outcome per field, review flag and reasons) and a summary file.
4. **Order.** B-allfast and B-allfull ran before the routing analysis; the derived thresholds were then used for B-tuned, which ran last.
5. **Quality check.** Every run was checked by its mean OCR time per invoice, which lies between about 0.55 and 0.63 s with three workers on an idle machine. Runs outside this range had been disturbed by other programs. They were moved to a `discarded/` folder, with the reason in the folder name, and repeated.

6. **Aggregation.** A report script combined the runs of each configuration into tables of mean ± standard deviation and produced the figures.

Every invocation creates a new output folder containing the command, all settings, the hardware and library versions, the log, the per-invoice records and the aggregated tables, so nothing is overwritten and every number can be traced to a file. The final benchmark is stored in `outputs/2026-10-18_23-52-41_full_benchmark`.

3.7 Additional experiments

The main benchmark submits all invoices at once and uses one OCR engine and one model family. Three additional experiments cover the questions it cannot answer (Table 3.7).

Table 3.7. Additional experiments.

Experiment	Question	Design
Latency	How long does a user wait when invoices arrive one at a time?	60 invoices (31 real, 29 synthetic) arriving as a Poisson process at 30, 48, 120 and 240 per minute; A, B as designed (waits for 8), B with dynamic batching; 3 runs per rate; cache off
OCR engines	Would a neural OCR engine be better?	Tesseract (CPU) and EasyOCR (GPU; JaidedAI, n.d.) read all 600 invoices once; each engine's text goes through the identical downstream of A
VLM baseline	Is an OCR stage still needed?	Qwen2-VL-2B-Instruct (Wang et al., 2024) in 4-bit NF4 reads only the image and answers in JSON; at most 1,024 visual tokens; same normalisation and scorer; a 20-invoice probe at 512, 1,024 and 2,048 tokens

The results are stored in `outputs/2026-10-20_01-01-33_latency_60docs`, `outputs/2026-10-20_01-52-39_ocr_comparison` and `outputs/2026-10-20_23-54-32_vlm_baseline`.

In the **latency experiment**, arrival times were drawn with a fixed seed per run and rate, so all three variants saw exactly the same arrivals. Latency is measured from an invoice's arrival to its result. In the **OCR comparison**, EasyOCR's text boxes go through the same row reconstruction as Tesseract's, so the experiment compares recognition rather than layout. Accuracy is measured twice: whether each true value appears anywhere in the OCR text (an upper bound for any extractor), and the end-to-end F1. With Tesseract, the end-to-end F1 reproduces A's benchmark value, which validates the experiment. In the **VLM baseline**, the model's 16-bit weights (4.4 GB) do not fit the GPU, so it was loaded in 4-bit. Each resolution setting ran in its own process, because an earlier probe that ran several settings in one process had included the previous model in the memory peaks.

3.8 Data analysis

- **Descriptive statistics.** Each configuration is summarised by the mean and standard deviation of its three runs, and comparisons are expressed as ratios of means. Differences of a few percent between configurations that should behave identically are treated as run-to-run variation and not interpreted.
- **Attribution.** The contribution of each optimisation is the change in wall time when only that factor is removed or altered (B-1worker, B-nobatch, B-nocache and the precision variants), relative to B.
- **Bottleneck model.** To explain the measured wall times, B is modelled as two overlapping stages, OCR on w workers and GPU work in one thread. With mean OCR time o and mean line-filter and model time $e + g$ per invoice, the predicted wall time for N invoices is

$$W_B \approx max(N \cdot o / w , N \cdot (e + g)) \quad (3.3)$$

- **Routing thresholds.** From B-allfast and B-allfull, each invoice's field accuracy under both routes is known. A grid of thresholds (T from 50 to 800 tokens, C from 70 to 95) is searched for the policy that

sends the most invoices to FAST while keeping field accuracy within 0.5 percentage points of all-FULL:

$$(T_{opt}, C_{opt}) = \text{argmax share}_{FAST}(T, C) \text{ subject to } acc(T, C) \geq acc_{FULL} - 0.005 \quad (3.4)$$

- **Error analysis.** Every wrong total of the first benchmark was examined for whether the correct amount had been present in the text given to the model, and in which position. This analysis motivated the iteration described in Section 3.1.
- **Estimates.** Two improvement options were not implemented but estimated by re-applying rules to the stored per-invoice predictions: stricter label checks in the review flag, and sending flagged invoices to the VLM. They are reported as estimates, separate from measurements.
- **Re-scoring.** Because all predictions are stored per invoice, a correction of the scorer can be applied without re-running any model. This was done once, after the OCR comparison revealed that dates written as "April 19,2024" were parsed with the current year. The parser was fixed and tested, and all stored runs were re-scored; the main benchmark did not change.

3.9 Validity, reliability and ethics

Internal validity. The architectures share every component that produces an answer and run on the same device, so differences cannot stem from different models or hardware paths. Warm-up excludes start-up effects. Decoding is greedy, so the same configuration produces the same answers in every run. Disturbed runs were detected through the OCR-time check and repeated; 33 runs were discarded in total, mostly because of two demonstration runs started during the benchmark and a runaway background process that occupied up to 8 of the 12 CPU threads.

Reliability. Three runs per configuration give a measure of the timing spread. Accuracy is deterministic: identical in all runs except with 8-bit weights, whose outlier handling varies slightly. The normalisation, scoring, validation and streaming scheduler are covered by 66 unit tests.

External validity. The study uses one small model family, three header fields on mostly English single-page invoices, and one laptop. The balance between the CPU stage and the GPU stage, and therefore which optimisation matters most, depends on the hardware; this is discussed in Chapter 5.

Construct validity. Cost is a model based on wall time and one GPU price; it ignores energy, CPU price and idle time. Because the main benchmark submits all invoices at once, its latency values describe queueing in a bulk job. Responsiveness is therefore measured separately with random arrivals. F1 is complemented by the share of fully correct invoices, which reflects what a user experiences.

Ethics and data protection. The real invoices are publicly available templates and sample scans and the rest are synthetic; no confidential company or personal data was used. All processing runs locally and no document is sent to an external service. Label corrections are documented, and the original labels are kept.

4 Findings

This chapter reports the findings, organised by research question. Section 4.1 gives an overview. Sections 4.2 and 4.3 answer RQ1 (performance and accuracy of A and B), Sections 4.4 to 4.6 answer RQ2 (which optimisations matter), and Sections 4.7 and 4.8 answer RQ3 (OCR engine and vision-language model). Section 4.9 reports further findings on data quality and measurement, and Section 4.10 summarises the findings per research question. Unless stated otherwise, values are the mean ± standard deviation of three runs over 600 invoices from `outputs/2026-10-18_23-52-41_full_benchmark`. Interpretation and implications follow in Chapter 5.

4.1 Overview

Table 4.1. Key findings at a glance.

Aspect	Finding
Throughput	B processes 301.7 invoices per minute, A 53.1 (5.7×)
Accuracy	identical: F1 0.941 in both; 505 of 600 invoices (84.2 %) fully correct in both
Memory	B uses 1.9× the GPU memory (1996 vs 1067 MB) and 1.4× the RAM
Cost	€0.0019 vs €0.0110 per 100 invoices (5.7× lower)
Sources of the gain	parallel OCR 2.8×, GPU batching 2.1×; B is limited by OCR
Quantisation	4-bit saves a third of the GPU memory but no time; fp16 is best; 8-bit is slowest
Latency	a fixed batch of 8 makes light-load latency worse than A (7.9 vs 1.7 s); dynamic batching gives 0.95–1.8 s at every load
Extraction	row-ordered OCR text and an arithmetic total check raised F1 from 0.85 to 0.94
OCR vs VLM	Tesseract beats EasyOCR here; a 2B-parameter VLM is more accurate (F1 0.966) but 19× slower than B

4.2 Performance of Architecture A and Architecture B (RQ1)

Table 4.2. Performance of A and B on 600 invoices.

Measure	A (sequential)	B (optimised)	B compared with A
Wall time for 600 invoices	677.9 ± 7.4 s	119.4 ± 4.7 s	5.7× faster
Throughput	53.1 ± 0.6 /min	301.7 ± 11.8 /min	5.7× higher
Service time per invoice	1.119 ± 0.012 s	0.718 ± 0.037 s	1.6× lower
of which OCR	0.588 s	0.576 s	equal
of which line filter	0.017 s	0.008 s	2× lower
of which LLM	0.514 s	0.134 s	3.8× lower
Pipeline GPU memory peak	1067 MB	1996 MB	1.9× higher
Whole-GPU memory peak	1388 MB	3172 MB	within 4 GB
Peak RAM, all processes	1438 MB	2012 MB	1.4× higher
Cost per 100 invoices	€0.0110	€0.0019	5.7× lower

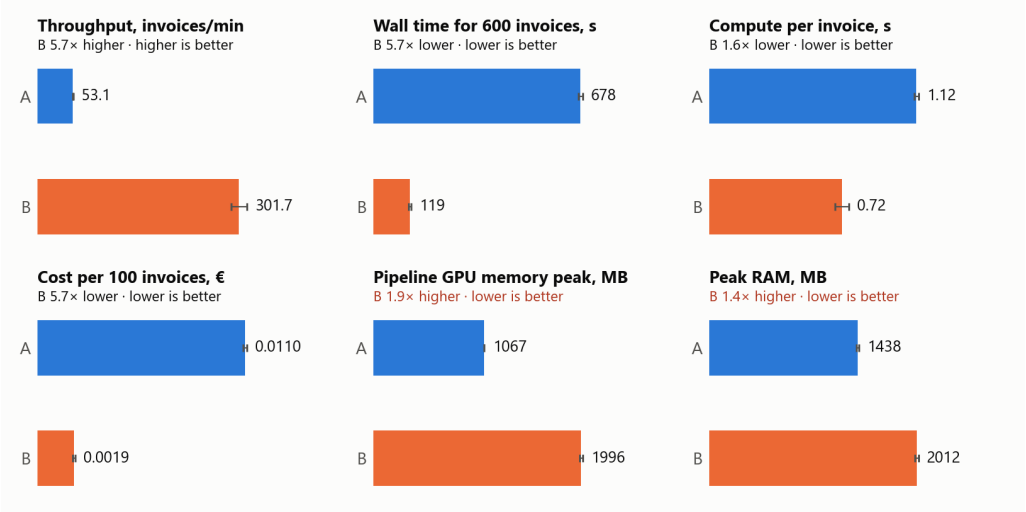


Figure 4.1. A and B on six performance measures (mean of 3 runs; whiskers: standard deviation). Red subtitles mark the measures on which B is worse.

Finding 1: B is 5.7 times faster and cheaper. B finishes the 600 invoices in 119.4 s instead of 677.9 s (Table 4.2, Figure 4.1). Throughput improves much more than the compute spent per invoice (1.6×). The difference is overlap: in B three OCR processes and the GPU work on different invoices at the same time, while A does one thing at a time. OCR costs the same per invoice in both architectures, and the LLM step is 3.8 times cheaper in B because 24 prompts share one call. Cost follows wall time and falls by the same factor.

Finding 2: B needs more memory, within the hardware limits. B keeps a 4-bit and an fp32 copy of the model and processes larger batches, so its pipeline GPU memory is 1.9 times A's. Its three OCR processes raise RAM by 40 %. The whole card peaks at 3.2 GB, below the 4 GB limit.

Finding 3: B's gain is in throughput, not in single-invoice speed. In run 1, B had finished all 600 invoices after 115 s; by then A had finished 142 (Figure 4.2). B's first result arrives after 4.4 s, A's after 0.6 s, because B first collects a batch of 8; a single invoice takes about 1.3 s in both.

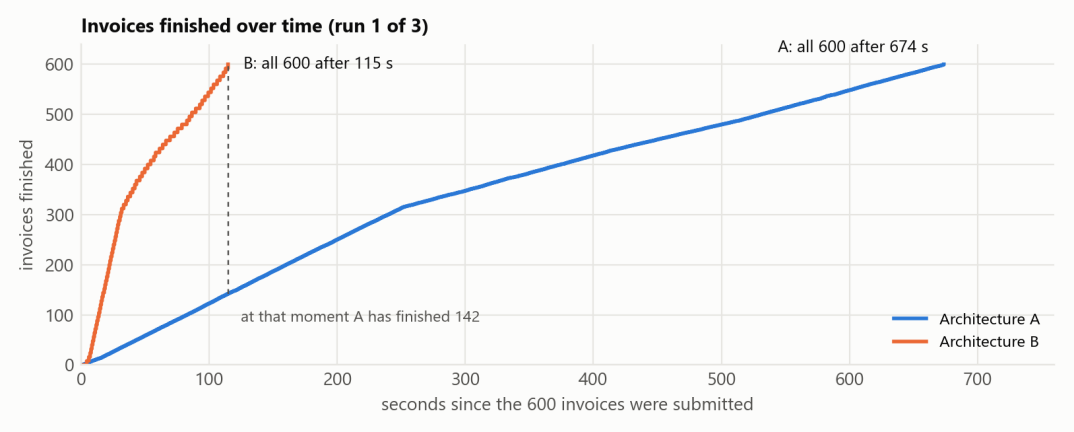


Figure 4.2. Invoices finished over time in run 1. The bend in both curves is where the real invoices start, which take longer to read than the synthetic ones.

Finding 4: with a fixed batch size, B is slower than A at light load; dynamic batching fixes this. When invoices arrive one at a time (`outputs/2026-10-20_01-01-33_latency_60docs`), A answers in 1.7 s at 30 invoices per minute, but its queue grows without bound above its capacity of about 53 per minute. B as designed waits until 8 invoices are read, which costs 7.9 s at light load and becomes faster as load increases. B with dynamic batching, where the GPU takes whatever has been read whenever it is free, has the lowest median and 95th-percentile latency at every rate tested (Figure 4.3, Table 4.3). F1 is identical in all 36 runs (0.944 on these 60 invoices).

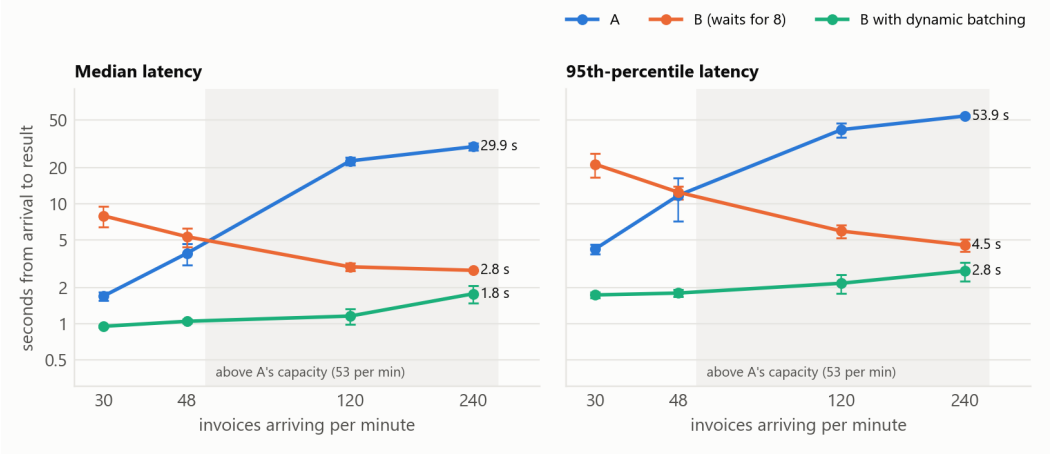


Figure 4.3. Latency from arrival to result for 60 invoices arriving at random times (mean of 3 runs; log scales). The grey area lies above A's capacity.

Table 4.3. Latency in seconds from arrival to result (mean ± standard deviation of 3 runs; p95 as mean).

Arrivals per min	A median	A p95	B median	B p95	Dynamic median	Dynamic p95
30	1.70 ± 0.13	4.19	7.94 ± 1.57	21.40	0.95 ± 0.03	1.74
48	3.87 ± 0.79	11.77	5.31 ± 0.93	12.46	1.05 ± 0.04	1.80
120	22.76 ± 1.64	41.46	2.98 ± 0.22	5.92	1.16 ± 0.18	2.18
240	29.94 ± 2.03	53.88	2.79 ± 0.02	4.52	1.78 ± 0.29	2.76

4.3 Accuracy of Architecture A and Architecture B (RQ1)

Finding 5: A and B are equally accurate. Both reach F1 0.941 over the 600 invoices, with 505 invoices (84.2 %) fully correct (Table 4.4). Accuracy is near perfect on the synthetic invoices and lower on the real ones, where the total is the hardest field. The small per-field differences between A and B come from B's 4-bit route and cancel out overall (Figure 4.4). Precision (0.937) and recall (0.945) are close, because the model almost never leaves a field empty.

Table 4.4. Accuracy of A and B (identical in all 3 runs).

Architecture	F1 all	F1 real	F1 synthetic	F1 number / date / total	Fully correct, real	Fully correct, all
A	0.941	0.892	0.989	0.968 / 0.960 / 0.894	71.7 %	84.2 %
B	0.941	0.894	0.987	0.967 / 0.960 / 0.896	72.3 %	84.2 %

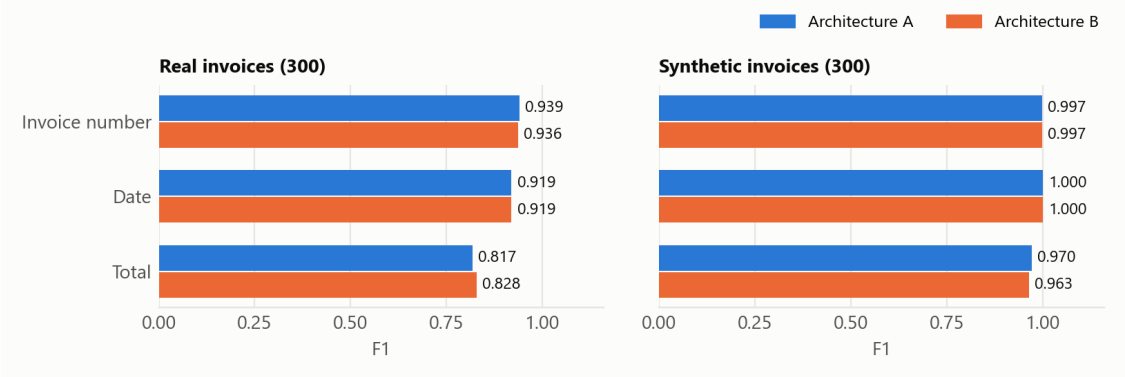


Figure 4.4. F1 per field for real and synthetic invoices in A and B (run 1).

Finding 6: the shared extraction stage, not the architecture, determined accuracy. The first complete benchmark reached only F1 0.85, with about a third of the totals on real invoices correct. In 154 of its 199 wrong totals the correct amount had been in the text given to the model. Tesseract's column-wise output had separated the label "Total" from its amount, and in summary rows such as `Total | $ 297,35 | $ 29,73 | $ 327,08` the model returned the net amount. Rebuilding the OCR text row by row and reconciling subtotal + tax = total raised F1 to 0.941 and the share of fully correct real invoices from 24 % to 72 % in both architectures (Table 4.5, Figure 4.5). The reconciliation changed the total on 148 invoices. Of the 146 totals that turned from wrong to right, 121 belong to invoices whose labels were never corrected, so the gain does not depend on the label corrections.

Table 4.5. Accuracy before and after the extraction upgrade.

Extraction	Arch.	F1 all	F1 real	F1 synth.	Real: number / date / total	Real fully correct
Block-order text, model's total	A	0.847	0.704	0.989	0.93 / 0.86 / 0.32	24 %
	B	0.850	0.712	0.987	0.93 / 0.86 / 0.34	26 %
Row-order text + total check	A	0.941	0.892	0.989	0.94 / 0.92 / 0.82	72 %
	B	0.941	0.894	0.987	0.94 / 0.92 / 0.83	72 %

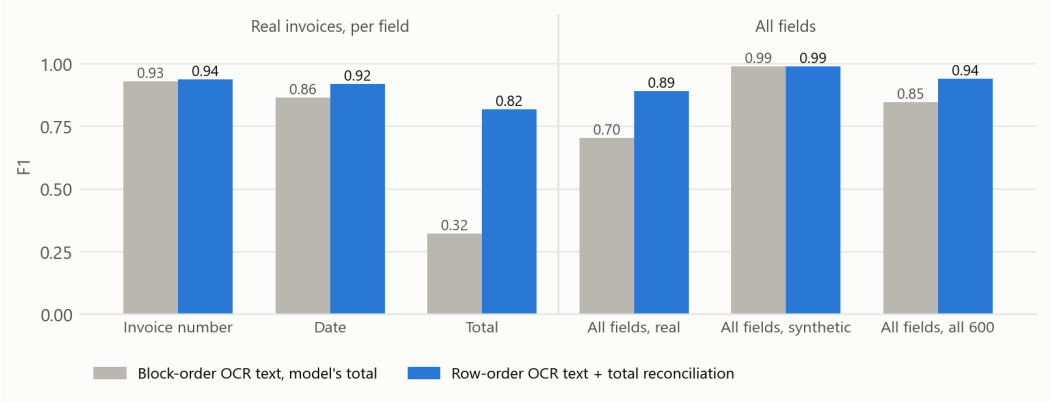


Figure 4.5. F1 of Architecture A before and after the extraction upgrade: per field on real invoices (left) and over all fields (right).

Finding 7: the review flag is precise but catches fewer than half of the errors. About 92 % of the invoices are accepted without review, and 90.5–90.9 % of those are fully correct, against 84.2 % if everything were accepted (Table 4.6). Over 90 % of the flagged invoices really were wrong. However, the flag finds only 42–45 of the 95 wrong invoices, because an invoice number or date copied from a neighbouring field passes every rule.

Table 4.6. Human review (run 1; identical in all runs).

	Architecture A	Architecture B
Accepted without review (straight-through)	92.0 %	92.5 %
Fully correct among accepted invoices	90.9 %	90.5 %
Flagged invoices that really were wrong	45 of 48 (94 %)	42 of 45 (93 %)
Wrong invoices caught by the flag	45 of 95 (47 %)	42 of 95 (44 %)

4.4 Sources of the speed-up (RQ2)

Table 4.7. Ablation of B: each variant changes one factor (mean ± standard deviation of 3 runs).

Configuration	Wall time for 600	Throughput	Effect
B (3 OCR workers, batch 8, cache, routing)	119.4 ± 4.7 s	301.7 /min	reference
B-1worker: no OCR parallelism	338.4 ± 5.1 s	106.4 /min	parallel OCR is worth 2.8×
B-nobatch: batch size 1	248.8 ± 2.5 s	144.7 /min	GPU batching is worth 2.1×
B-6workers	102.9 ± 10.3 s	352.1 /min	16 % faster, largest spread, 2.6 GB RAM
B-nocache: no cache	126.0 ± 2.6 s	285.9 /min	the cache lookup costs nothing measurable

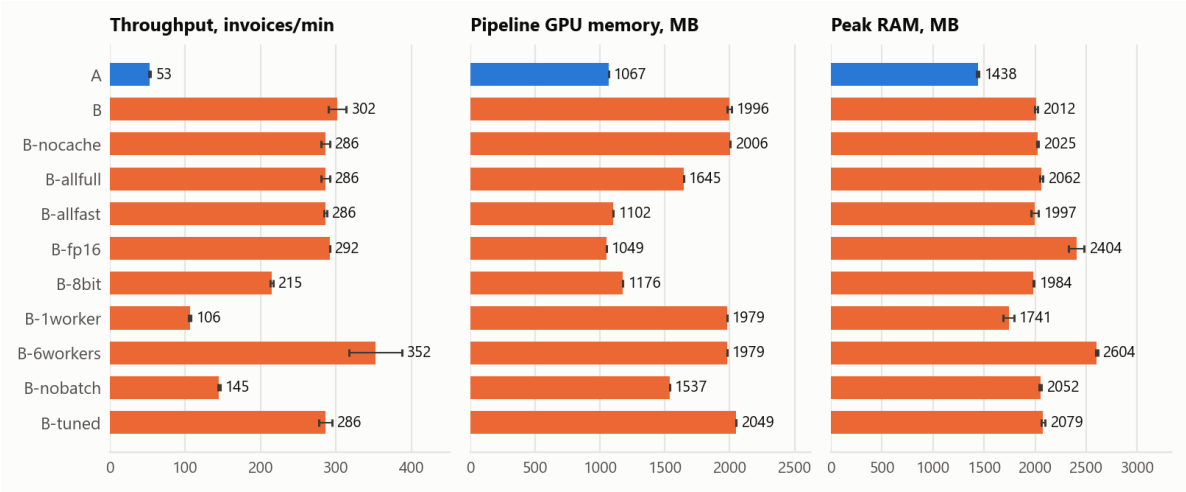


Figure 4.6. Throughput, pipeline GPU memory and peak RAM of every configuration (mean of 3 runs; whiskers: standard deviation). B-warm (0.59 s for all 600 invoices) is omitted.

Finding 8: parallel OCR and GPU batching produce the speed-up. Removing OCR parallelism makes B 2.8 times slower, and removing batching 2.1 times slower (Table 4.7, Figure 4.6). Doubling the OCR workers to six gains only 16 % more, with the largest run-to-run spread, because six Tesseract processes slow each other down on 12 logical cores (OCR time per invoice rises from 0.58 to 0.76 s). Configurations that should behave identically on a first pass differ by up to 5 %, which is the run-to-run variation of the laptop.

Finding 9: B is OCR-bound, and a two-stage bottleneck model predicts every variant. Equation 3.3 predicts B's wall time from its mean stage times within 1–8 % for all ten variants (Table 4.8, Figure 4.7). With three workers OCR is the slower stage. Without batching, with 8-bit weights, or with six workers, the GPU becomes the slower stage. This explains why a faster model step barely shortens B's total time.

Table 4.8. Bottleneck model: time each stage needs for 600 invoices, predicted and measured wall time.

Configuration	OCR stage	GPU stage	Predicted	Measured	Slower stage
B	115.1 s	85.2 s	115.1 s	119.4 s	OCR
B-1worker	334.8 s	79.1 s	334.8 s	338.4 s	OCR
B-6workers	76.2 s	94.8 s	94.8 s	102.9 s	GPU
B-nobatch	117.6 s	239.6 s	239.6 s	248.8 s	GPU
B-fp16	120.7 s	53.5 s	120.7 s	123.5 s	OCR
B-8bit	126.0 s	159.8 s	159.8 s	167.8 s	GPU
B-allfast	122.8 s	75.1 s	122.8 s	125.8 s	OCR

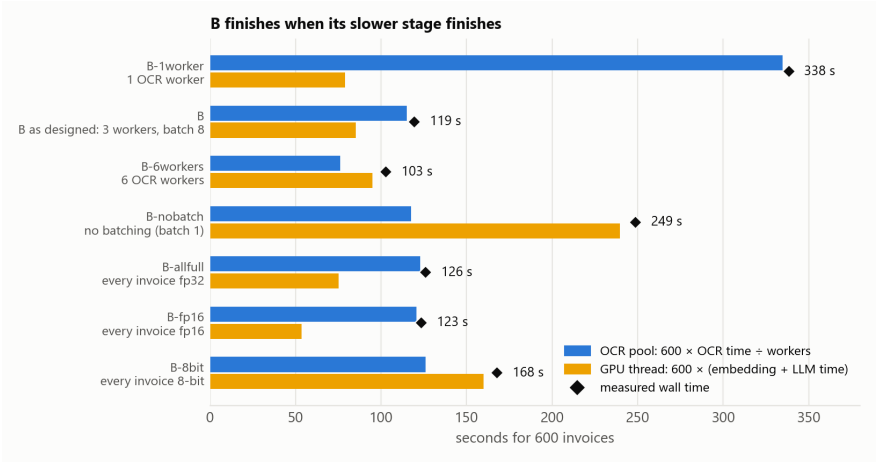


Figure 4.7. Time each stage of B needs for 600 invoices (from the mean stage times) and the measured wall time.

4.5 Model precision and quantisation (RQ2)

Table 4.9. B with every invoice on one model, in four precisions.

Precision	Wall time for 600	LLM time per invoice	Pipeline GPU peak	F1
fp32 (B-allfull)	126.1 ± 2.6 s	0.115 s	1645 MB	0.941
fp16 (B-fp16)	123.5 ± 0.1 s	0.079 s	1049 MB	0.941
4-bit NF4 (B-allfast)	125.8 ± 0.8 s	0.115 s	1102 MB	0.941
8-bit LLM.int8 (B-8bit)	167.8 ± 1.6 s	0.258 s	1176 MB	0.9385 ± 0.0003

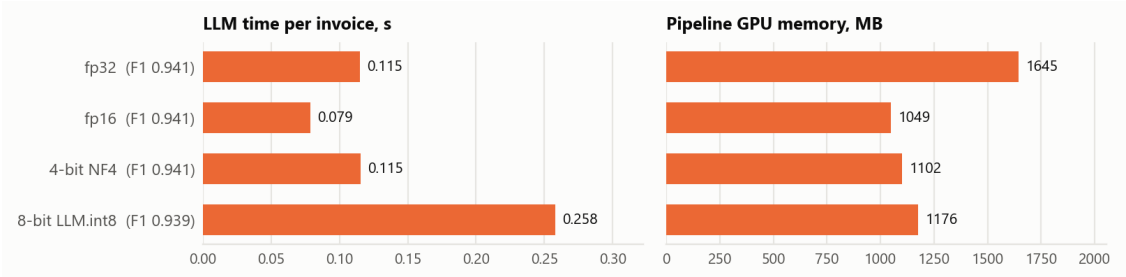


Figure 4.8. LLM time per invoice (left) and pipeline GPU memory (right) of B with every invoice on one model.

Finding 10: 4-bit quantisation saves memory, not time. The NF4 model needs a third less GPU memory than fp32 at the same LLM time and the same F1 (Table 4.9, Figure 4.8).

Finding 11: half precision is the best option on this GPU, and 8-bit the worst. fp16 has the fastest model step (0.079 s), the least memory and the same accuracy. LLM.int8 makes the model step 2.2 times slower than fp32, and it is the only precision whose accuracy varies between runs (F1 0.9385 ± 0.0003). Because B is OCR-bound, even fp16's 31 % faster model step shortens B's wall time by only about 2 %.

4.6 Routing and cache (RQ2)

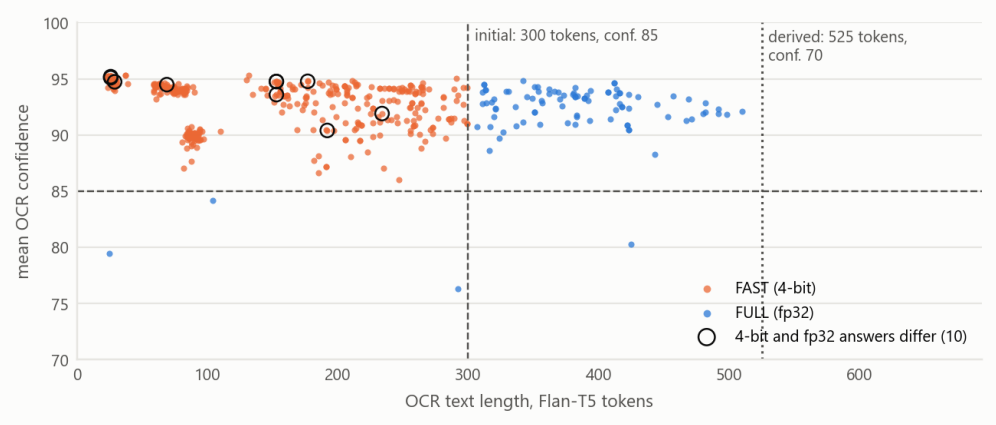


Figure 4.9. Routing per invoice with the initial thresholds (dashed: 300 tokens, confidence 85) and the derived token threshold (dotted: 525). Circles mark the 10 invoices where the 4-bit and fp32 answers differ; one invoice with OCR confidence below 70 is not shown.

Finding 12: the router works, but two precisions of one small model leave it little to choose between. With the initial thresholds, B sends 505 invoices to the 4-bit route and 95 to the fp32 route (Figure 4.9). Both routes reach the same field accuracy (0.936) and disagree on only 10 invoices, 5 in each direction. Accuracy is equal within 0.5 percentage points in every token-count bin (Figure 4.10) and every confidence bin. The data-derived thresholds are therefore T = 525 tokens and C = 70, which send 99.8 % of invoices to the 4-bit route. Three fresh runs with these thresholds (B-tuned) confirm F1 0.941 at 126.1 ± 3.7 s.

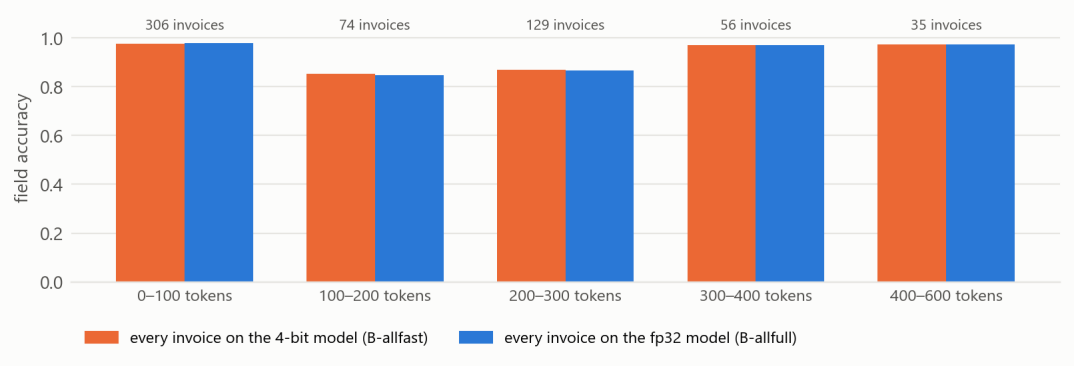


Figure 4.10. Field accuracy per OCR token-count bin with every invoice on the 4-bit model and on the fp32 model.

Finding 13: the cache helps only repeated invoices, and costs nothing otherwise. Submitting the same 600 invoices again takes 0.59 ± 0.03 s, about 1 ms per invoice, against 119.4 s on the first pass (Table 4.10). On a first pass, B with the cache was not slower than B without it.

Table 4.10. First and repeated pass of the same 600 invoices.

Run	Wall time for 600	Cache hits	Per invoice
B, first pass (cache empty)	119.4 ± 4.7 s	0	199 ms
B-warm, the same invoices again	0.59 ± 0.03 s	600	≈ 1 ms
B-nocache, first pass without cache	126.0 ± 2.6 s	–	210 ms

4.7 OCR engine (RQ3)

Table 4.11. Tesseract and EasyOCR on all 600 invoices with the same downstream and scorer (`outputs/2026-10-20_01-52-39_ocr_comparison`).

Engine	Runs on	OCR time per invoice, mean / p95	GPU memory	F1 all	F1 real	F1 synth.	Fully correct
Tesseract 5.4	CPU, 1 thread	0.57 / 1.22 s	none	0.941	0.892	0.989	84.2 %
EasyOCR 1.7.2	GPU	4.16 / 13.49 s	3883 MB	0.934	0.871	0.997	82.7 %

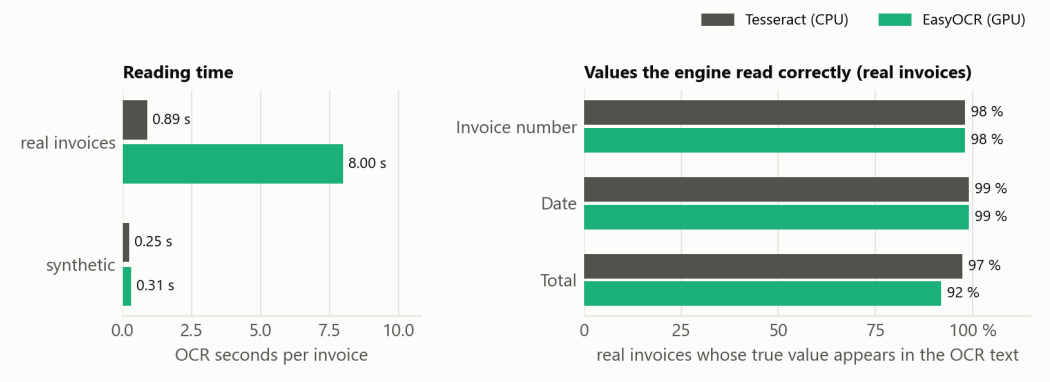


Figure 4.11. Left: OCR time per invoice. Right: share of real invoices whose true value appears anywhere in the OCR text.

Finding 14: Tesseract is the better OCR engine for this pipeline. It is slightly more accurate (F1 0.941 vs 0.934; real invoices 0.892 vs 0.871) and nine times faster on the real scans (0.89 vs 8.0 s per invoice), although EasyOCR runs on the GPU (Table 4.11, Figure 4.11). EasyOCR's memory peak of 3.9 GB fills almost the whole card, leaving no room for the language model, while Tesseract needs no GPU memory at all. The accuracy gap lies in the total: the true total appears in Tesseract's text for 97 % of the real invoices and in EasyOCR's for 92 %. On the clean synthetic invoices EasyOCR is slightly better (0.997 vs 0.989).

4.8 Vision-language model on the same GPU (RQ3)

Table 4.12. OCR pipelines and a VLM on the same 600 invoices and scorer (`outputs/2026-10-20_23-54-32_vlm_baseline`).

System	Time per invoice	GPU peak	F1 all	F1 real	Real: number / date / total	Fully correct
A (OCR text)	1.13 s	1067 MB	0.941	0.892	0.94 / 0.92 / 0.82	505
B (OCR text)	0.20 s	1996 MB	0.941	0.894	0.94 / 0.92 / 0.83	505
Qwen2-VL-2B, 4-bit, ≤ 1,024 visual tokens (image)	3.90 s	1745 MB	0.966	0.930	0.99 / 0.96 / 0.85	541

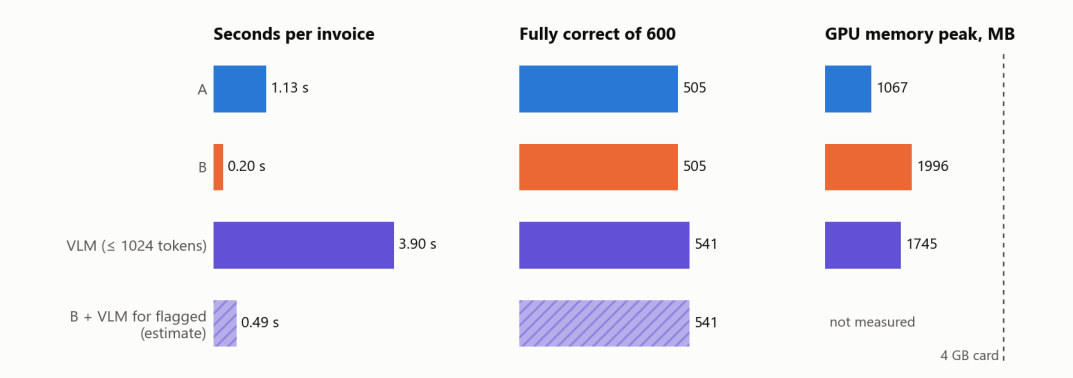


Figure 4.12. Time per invoice, fully correct invoices and GPU memory. The hatched row is an estimate: B's answers, with the VLM's answers for the invoices that B flags for review.

Finding 15: the VLM is more accurate but 19 times slower than B. Reading the image directly, Qwen2-VL-2B reaches F1 0.966 and 541 fully correct invoices, against 505 for the OCR pipelines. Its advantage is largest for invoice numbers and dates (Table 4.12, Figure 4.12). It needs 3.9 s per invoice (15.4 invoices per minute), 19 times B's time and 3.5 times A's, and it runs on the 4 GB card only in 4-bit. It produced valid JSON for 99.7 % of the invoices.

Finding 16: resolution trades memory and time, with no accuracy gain beyond 1,024 visual tokens. On a 20-invoice probe with one setting per process, the VLM needed 3.2 s and 1568 MB at 512 visual tokens, 3.8 s and 1745 MB at 1,024, and 5.2 s and 2329 MB at 2,048. The highest F1 on this sample (1.00, against 0.95 at the other two settings) was at 1,024 tokens. The sample is small, so this only shows that more resolution did not help further.

Finding 17 (estimate): the VLM is most useful for the invoices that B flags. On the 45 invoices that B sends to review, B is fully correct 3 times and the VLM 39 times. Applying the VLM only to these, estimated from the stored predictions, gives 541 fully correct invoices, as many as the VLM alone, for about 0.29 s of extra GPU time per invoice on average, roughly 1/13 of running the VLM on every invoice.

4.9 Further findings on data quality and measurement

- **Ground truth.** 26 of the 300 real-invoice labels held the net amount instead of the amount due; they were corrected under the net + tax = gross rule. Inconsistent labels are a realistic risk in invoice datasets and directly affect measured accuracy.
- **Scorer error.** Dates written without a space after the comma ("April 19,2024") had been parsed with the current year. The error surfaced only in the OCR comparison, because EasyOCR writes dates that way. After the fix and re-scoring, EasyOCR's F1 rose from 0.852 to 0.934, while the main benchmark was unaffected.
- **Measurement disturbances.** 33 runs were disturbed by other programs: two demonstration runs, one of which cleared the shared cache file, and a runaway background process. All were detectable through the OCR time per invoice, which rose from about 0.6 s to about 0.95 s, and they were repeated. Wall-clock benchmarks on a laptop therefore need a per-stage sanity check for every run.
- **Label checks (estimate).** Requiring the invoice number and date to appear on a line with a matching label would let the review flag catch 64 instead of 42 of the 95 wrong invoices. Accepted invoices would then be fully correct in 94.0 % instead of 90.5 % of cases, at a straight-through rate of 86.2 % instead of 92.5 %.

4.10 Summary of findings by research question

Table 4.13. Answers to the research questions.

RQ	Answer from the findings
RQ1	B reaches 5.7× the throughput and 5.7× lower cost of A, with identical F1 (0.941), at 1.9× the GPU memory and 1.4× the RAM. Latency depends on the batching policy: a fixed batch of 8 is slower than A at light load, while dynamic batching is the fastest at every load tested (Findings 1–7).
RQ2	Parallel OCR (2.8×) and GPU batching (2.1×) produce the gain; their overlap turns a 1.6-fold reduction in compute into a 5.7-fold gain in throughput. B is OCR-bound, as a two-stage model predicts within 8 %. 4-bit quantisation saves memory but no time, fp16 is best and 8-bit is worst; the cache helps only repeated invoices; routing between two precisions of one small model changes little (Findings 8–13).
RQ3	Tesseract is faster and more accurate than EasyOCR here. A 2B-parameter VLM is more accurate (F1 0.966) but 19× slower than B; on this hardware the OCR pipeline wins on throughput, cost and memory headroom, and the VLM is best used for the invoices the pipeline flags (Findings 14–17).

References

Chung, H. W., Hou, L., Longpre, S., et al. (2022). *Scaling instruction-finetuned language models* (arXiv:2210.11416). arXiv.

Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*.

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

JaidedAI. (n.d.). *EasyOCR* [Software repository]. GitHub. https://github.com/JaidedAI/EasyOCR

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. Section 8.3. https://nlp.stanford.edu/IR-book/

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019* (pp. 3982–3992). https://doi.org/10.18653/v1/D19-1410

Smith, R. (2007). An overview of the Tesseract OCR engine. In *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)* (Vol. 2, pp. 629–633). IEEE. https://doi.org/10.1109/ICDAR.2007.4376991

Wang, P., Bai, S., Tan, S., et al. (2024). *Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution* (arXiv:2409.12191). arXiv.

Wang, W., Wei, F., Dong, L., et al. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.